Lecture 03.01: Conditional Statements in Python

📌 Executive Summary

Deep exploration of control flow branching, if/elif/else syntax, truth value testing, Boolean lattices, and CPython bytecode execution.

🔬 Theoretical Foundations & Principles

1. The Anatomy of Conditional Branching

In imperative programming, program execution normally proceeds sequentially down the instruction stream. Conditional statements introduce branch points, directing control flow along alternative execution paths based on truth-value evaluation.

The fundamental grammar in Python:

```
if condition_1:
    # Suite 1 executed if condition_1 evaluates to True
    statement_block_1()
elif condition_2:
    # Suite 2 executed if condition_1 is False AND condition_2 is True
    statement_block_2()
else:
    # Fallback suite executed when all preceding conditions evaluate to False
    fallback_statement_block()
```

2. Mathematical Formalization of Decision Boundaries

Let $\mathcal{X}$ denote the state space of program variables. A condition is a Boolean-valued decision function:

$$f : \mathcal{X} \to \{0, 1\}$$

The execution path P selected by an ordered chain of predicates $(f_1, f_2, \ldots, f_k)$ is partitioned as:

$$P = \begin{cases} 1 & \text{if } f_1(\mathbf{x}) = 1 \\ 2 & \text{if } f_1(\mathbf{x}) = 0 \land f_2(\mathbf{x}) = 1 \\ \vdots & \vdots \\ k+1 & \text{if } \bigwedge_{j=1}^{k} (f_j(\mathbf{x}) = 0) \end{cases}$$

3. Truth Value Testing & Boolean Type Coercion

Python evaluates any object in a Boolean context via its `__bool__()` or `__len__()` method.

- **Falsy Values:** `None`, `False`, numeric zeros (`0`, `0.0`, `0j`), empty collections (`""`, `()`, `[]`, `{}`, `set()`, `range(0)`).
- **Truthy Values:** All non-zero numbers, non-empty collections, and user-defined instances without a zero-returning `__bool__()` method.

💻 Production Implementation & Code

```
# Production Conditional Demonstration
def evaluate_admission_status(score: float, age: int) -> str:
    """Determines university admission qualification based on score and age."""
    if score >= 90.0:
        if age >= 18:
            return "DIRECT_ADMISSION_HONORS"
        else:
            return "PROVISIONAL_ADMISSION_UNDERAGE"
    elif score >= 75.0:
        return "STANDARD_ADMISSION"
    elif score >= 60.0:
        return "WAITLIST_REVIEW"
    else:
        return "ADMISSION_REJECTED"

print("Score 92, Age 20:", evaluate_admission_status(92, 20))
print("Score 84, Age 19:", evaluate_admission_status(84, 19))
print("Score 52, Age 21:", evaluate_admission_status(52, 21))
```

⚙️ Algorithmic Complexity & Systems Analysis

- **Time Complexity:** Detailed in respective algorithmic operations.
- **Auxiliary Space:** Memory bounds and CPython frame management.